A Coarse-Grain Governance Layer for Domain-Specific AI: Knowledge Maturation, Residual Control, and Expert Superiority Review

Part 1 — Abstract, Reader Contract, and Foundations

---

0. Abstract

The current trajectory of generative AI is moving through a structural transition. The first phase was dominated by monolithic generalist models: ever-larger systems trained on broad Internet-scale data and deployed as universal assistants. That trajectory produced impressive fluency, factual recall, and broad task coverage, but it also exposed three increasingly visible limits: high inference cost, weak abstraction in domains without formal structure, and difficulty producing deeply verifiable reasoning. A competing trajectory now emphasizes domain-specific superintelligence: smaller specialist systems trained on high-quality domain data, grounded in explicit abstractions such as knowledge graphs, ontologies, formal languages, and verification environments. This route is attractive because it aligns reasoning depth with domain structure and can reduce energy, latency, and deployment cost.

However, domain-specific AI by itself is not enough. A society of specialist models may still suffer from immature knowledge, hidden residuals, overconfident synthesis, opaque routing, and expert answers that sound sophisticated without proving that they are better than a simpler professional common-sense judgment. This paper proposes a complementary architecture: a coarse-grain governance layer for domain-specific AI. The layer is not another expert model. It is a professional common-sense envelope that sits outside mature domain systems and produces an explicit baseline judgment. Specialist outputs must then confirm, refine, or outperform this baseline through an expert superiority review.

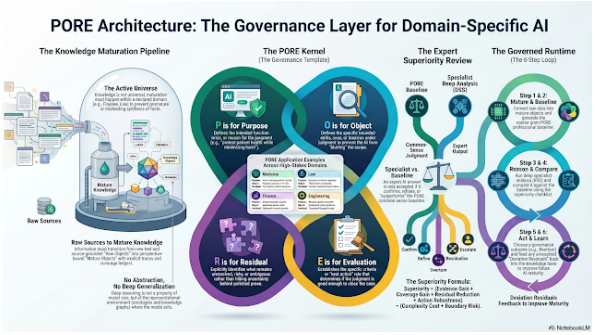
The proposed framework combines four components:

1. Knowledge maturation: raw sources are transformed into raw knowledge objects, then into mature universe-bound knowledge objects.
2. Residual control: unresolved ambiguity, contradiction, fragility, and coverage gaps are preserved as governable residuals rather than erased by polished synthesis.
3. PORE coarse-grain judgment: mature domain knowledge is projected into a compact professional judgment template: Purpose, Object, Residual, and Evaluation.
4. Expert superiority review: specialist conclusions must explain why they improve on the PORE baseline, using evidence gain, coverage gain, residual reduction, action robustness, complexity cost, and boundary risk.

The main thesis is simple:

Specialist AI should not only be accurate; it should be able to explain why its expert answer is better than the best coarse-grain professional common-sense answer. (0.1)

This shifts AI system design from answer generation toward governed judgment. The result is not a single giant mind, nor merely a society of expert agents, but a layered runtime in which mature knowledge, residual honesty, specialist reasoning, and executive common sense are kept distinct and forced into productive comparison.



---

1. Reader Contract and Scope

This paper is written for AI engineers, knowledge-system designers, domain experts, and technically serious readers interested in the next step beyond general-purpose large language models. The argument assumes familiarity with modern LLMs, retrieval-augmented generation, tool use, knowledge graphs, and agent orchestration, but it does not assume prior knowledge of PORE, SMFT, or any private terminology.

The term PORE will be introduced from scratch as a minimal coarse-grain judgment grammar. In this paper, PORE stands for:

Purpose / Object / Residual / Evaluation. (1.1)

It is not presented as a metaphysical theory. It is not a replacement for expert knowledge. It is not a theorem prover, ontology language, or new neural architecture. It is a compact governance template for asking four high-level questions:

What purpose or objective is being served?

What object, case, system, or decision is under judgment?

What residual remains unresolved under the current view?

What evaluation, test, or action criterion should govern the next step?

The central idea is that many professional judgments, especially at the leadership or executive level, do not begin with maximal technical detail. They begin with a coarse but disciplined sense of purpose, object, risk, and test. A good leader, entrepreneur, senior engineer, doctor, lawyer, or investigator often first asks:

What are we trying to achieve?

What exactly are we judging?

What obvious uncertainty or danger remains?

How would we know whether this is good enough?

The PORE layer formalizes this first-pass professional common-sense judgment. It is deliberately coarse-grained. Its function is not to replace specialists, but to create a baseline that specialists must beat.

The paper's proposal can be summarized as follows:

PORE_Baseline = CoarseGrain(Purpose, Object, Residual, Evaluation | MatureKnowledge). (1.2)

Expert_Answer is acceptable only if it confirms, refines, or outperforms PORE_Baseline. (1.3)

The important word is outperforms. The expert answer may be more technical, more precise, more domain-specific, or more counterintuitive. But if it differs from the professional common-sense baseline, it must explain why the difference is justified. It must show what the baseline missed, what evidence changes the judgment, what residual is reduced, and what new complexity or risk is introduced.

This is the paper's central governance move.

---

2. Why Another Layer Is Needed

2.1 The limit of monolithic generalist AI

The dominant generative AI paradigm has been to scale large generalist models. The expectation has been that larger parameter counts, more data, and more inference-time compute will gradually produce stronger reasoning across all domains. This has worked remarkably well for many forms of language use, summarization, coding assistance, factual recall, and broad conversational assistance.

But a growing body of research and engineering experience suggests that scale alone does not reliably produce robust compositional reasoning, especially in domains where the relevant abstractions are not already explicit in the training distribution. The Princeton paper argues that current LLMs show real reasoning depth most clearly in domains such as mathematics and coding, where rigorous abstractions already exist, and that domains lacking such abstractions remain harder for scale-driven LLMs to master reliably. It proposes domain-specific superintelligence as an alternative built on high-quality data, explicit symbolic abstractions, synthetic curricula, small specialist models, and societies of domain experts.

This is an important shift. It says that reasoning is not merely a property of a large model. It is also a property of the representational environment in which the model learns and acts.

A useful slogan is:

No abstraction, no deep generalization. (2.1)

More cautiously:

Weak abstraction implies fragile compositional generalization. (2.2)

This does not mean that neural models cannot learn abstractions. It means that, in many practical domains, the system should not merely hope that the right abstractions will emerge from scale. It should engineer them explicitly.

2.2 The DSS answer: specialist depth instead of monolithic breadth

Domain-specific superintelligence, or DSS, is a compelling response to the scaling bottleneck. The basic idea is to stop asking one model to be an expert in everything. Instead, build smaller specialist systems that are deeply grounded in one domain.

A DSS system may include:

- a domain-tuned small language model;
- a curated corpus of authoritative sources;
- a knowledge graph or ontology;
- formal rules, proof systems, or executable constraints;
- retrieval and verification tools;
- synthetic curricula generated from the domain abstraction;
- expert evaluation protocols.

The Princeton framework describes DSS societies: a front-end orchestrator decomposes a complex user query and routes sub-queries to specialist DSS backends. For example, a complex industrial accident might require an engineering DSS, a legal DSS, a safety-regulation DSS, and a financial-risk DSS. The final answer is then synthesized from specialist outputs.

This is a strong architectural direction. It aligns with how human organizations work. Difficult problems are rarely solved by one all-purpose thinker. They are solved by specialist teams, each with different knowledge, tools, standards, and failure modes.

However, DSS societies introduce a new problem.

A society of specialists is not automatically a governed judgment system. (2.3)

Specialists can disagree. Specialists can overfit. Specialists can produce answers that are locally impressive but globally misaligned. Specialist systems can become opaque if their routing, evidence, residuals, and confidence are not recorded in a structured way. Even worse, a specialist answer may sound superior merely because it is more technical.

This is why a coarse-grain governance layer is needed.

2.3 The missing professional common-sense baseline

In high-quality human institutions, expert analysis usually exists in tension with common-sense judgment. A senior leader, judge, project owner, physician, auditor, or investor often asks for expert advice, but does not accept expert complexity automatically. The expert must explain why the deeper view changes the first-pass view.

This institutional pattern is important.

A specialist conclusion is stronger when it can defeat a disciplined common-sense baseline. (2.4)

The baseline may be wrong. It may be crude. It may miss hidden variables. But precisely because it is simple, it gives the organization a comparison surface. The expert must answer:

Why is the obvious answer insufficient?

What hidden condition matters?

What evidence changes the decision?

Why is the additional complexity worth carrying?

What risk is reduced?

What risk is introduced?

Most current AI systems do not formalize this comparison. They either generate a direct answer or synthesize several expert-like outputs into a polished response. This can produce fluent but weakly governed judgment. The user receives a conclusion without seeing whether it genuinely improves on a competent first-pass professional view.

The proposed PORE layer fills this gap.

It produces a professional common-sense baseline before specialist outputs are accepted as final. Then it forces a review of superiority, deviation, and residual.

---

3. Background: PORE as a Coarse-Grain Judgment Grammar

3.1 What PORE is

PORE is a minimal template for structured professional judgment. It has four coordinates:

P = Purpose. (3.1)

O = Object. (3.2)

R = Residual. (3.3)

E = Evaluation. (3.4)

Together:

PORE := (P, O, R, E). (3.5)

Each coordinate answers one fundamental governance question.

Purpose asks: what is the intended function, value, target, or reason for judging?

Object asks: what bounded entity, case, claim, system, process, or decision is being judged?

Residual asks: what remains unresolved, risky, ambiguous, conflicting, fragile, or outside current coverage?

Evaluation asks: what criterion, test, decision rule, or next action will determine whether the judgment is good enough?

PORE is deliberately simple. Its power comes from being hard to avoid. Every serious decision needs some answer to these four questions. If one of them is missing, the judgment is usually unstable.

A decision without Purpose drifts. (3.6)

A decision without Object blurs. (3.7)

A decision without Residual lies. (3.8)

A decision without Evaluation cannot close. (3.9)

This is why PORE is suitable as a coarse-grain governance layer. It does not claim to contain all domain knowledge. It asks whether the domain knowledge has been compressed into a usable judgment form.

3.2 PORE is not a replacement for domain expertise

PORE should not be misunderstood as a shortcut around specialist reasoning. A PORE baseline is not the final answer in a high-stakes domain. It is a disciplined starting point.

For example, in medicine:

Purpose: protect patient health while minimizing harm.

Object: this patient's symptom cluster, lab results, and clinical history.

Residual: missing tests, uncertainty between competing diagnoses, risk of rare but serious conditions.

Evaluation: differential diagnosis quality, guideline consistency, safety of next action, need for escalation.

This does not replace a doctor. It gives a structured first-pass professional frame that a medical DSS or human physician must refine.

In law:

Purpose: resolve a dispute under applicable authority and procedural constraints.

Object: the legal issue, fact pattern, jurisdiction, and claim type.

Residual: uncertain facts, conflicting precedents, missing evidence, interpretive ambiguity.

Evaluation: controlling authority, burden of proof, litigation risk, remedy viability.

This does not replace legal reasoning. It gives a baseline that legal experts must either confirm or overturn.

In engineering:

Purpose: deliver a working system under safety, cost, and reliability constraints.

Object: the component, process, failure mode, or design decision.

Residual: unknown root cause, untested edge cases, hidden dependencies, reliability gaps.

Evaluation: test results, failure reproduction, safety margin, maintainability, cost of repair.

Again, PORE does not solve the engineering problem. It frames the professional common-sense view that specialist analysis must beat.

3.3 PORE as coarse-graining

The word coarse-grain is important. In physics, statistics, and systems engineering, coarse-graining does not mean careless simplification. It means compressing a high-dimensional state into fewer effective variables that preserve the dominant behavior relevant to the current level of control.

In this paper, PORE is a coarse-graining operator over mature domain knowledge.

PORE_Card = C_PORE(K_mature, Q, U, Res, Cov). (3.10)

where:

K_mature = mature domain knowledge objects. (3.11)

Q = current query or decision context. (3.12)

U = active universe or domain perspective. (3.13)

Res = known residual packets. (3.14)

Cov = coverage ledger. (3.15)

The output is not a full expert proof. It is a professional common-sense card:

PORE_Card := {Purpose, Object, Residual, Evaluation, BaselineJudgment, FirstInvestigationPath}. (3.16)

This is the first major design rule:

The PORE layer should not directly consume raw sources when mature objects are available. (3.17)

If the PORE layer reads raw text directly, it risks becoming a shallow summarizer. Its proper role is different. It should operate after knowledge maturation has already produced reliable domain objects, coverage records, and residual packets.

This makes the PORE layer a governance wrapper, not another retrieval model.

---

4. Knowledge Maturation Before PORE

4.1 Raw text is not mature knowledge

A central weakness in many LLM systems is the assumption that retrieving relevant text is enough. A document chunk may contain useful evidence, but a chunk is not yet a stable knowledge object. A wiki page may summarize a source, but a page is not necessarily mature knowledge. A knowledge graph triple may encode a relation, but the triple may still be fragile, incomplete, or insufficiently contextualized.

The knowledge maturation view says that knowledge should move through stages:

Raw Source -> Raw Object -> Mature Object -> Runtime-Usable Knowledge. (4.1)

A raw source is a document, note, paper, manual, transcript, table, legal case, policy, or dataset.

A raw object is a source-grounded extraction that preserves provenance and local structure.

A mature object is a consolidated domain object, formed under a declared perspective or universe, with coverage, trace, residuals, and update history.

The Knowledge Object architecture frames raw pages as immature concept attractor objects and mature pages as consolidated, perspective-bound concept attractor objects. It emphasizes that a persistent wiki is not yet a governed knowledge runtime; mature knowledge requires trace, residual handling, coverage ledgers, and universe-bound assimilation.

This matters because a PORE baseline should not be based on unstable source fragments. It should be based on mature knowledge objects.

4.2 Why the active universe matters

Knowledge is not always universal. A fact pattern can mean different things in different domains. A financial event may be interpreted one way by accounting, another way by legal, another way by operations, and another way by risk management. Each perspective has its own admissible objects, causal assumptions, evaluation criteria, and residual types.

Therefore, maturation should be universe-bound.

K_mature(U) = Assimilate(U, K_raw, Schema, Trace, Residual, Coverage). (4.2)

where U is the declared active universe.

This prevents premature universalization. It also prevents a common LLM failure: blending perspectives too early into a fluent but poorly governed synthesis.

For example, a product delay may be:

- an engineering dependency problem;
- a customer communication problem;
- a contract penalty problem;
- a revenue recognition problem;
- a leadership credibility problem.

A mature knowledge system should not collapse these into one generic summary too soon. It should preserve the universe in which each judgment is made.

Only after this maturation can the PORE layer ask:

Under this active universe, what is the professional common-sense baseline? (4.3)

4.3 Coverage as a missing state variable

A domain-specific AI system should not only know what it knows. It should know what portion of the relevant domain has been covered, what remains outside closure, and what has been deliberately excluded.

Coverage is therefore a governance state.

Coverage(K, U) = absorbed_structure + unresolved_residual + excluded_scope. (4.4)

Without coverage, an AI system may produce a polished answer from partial evidence. With coverage, it can say:

This baseline is strong because the main mature objects are covered.

This baseline is weak because key residuals remain unresolved.

This expert answer is superior because it covers a missing region.

This answer should be escalated because the active universe is under-covered.

The PORE layer should always carry a coverage warning.

A PORE baseline without coverage status is only rhetoric. (4.5)

---

5. Residual Control as the Honesty Layer

5.1 The bounded observer problem

No AI system, human expert, or organization sees the whole world at once. Every observer is bounded by compute, time, memory, representation, tools, training, institutional incentives, and admissible actions.

A bounded observer extracts some stable structure and leaves some remainder unresolved.

MDL_T(X) = S_T(X) + H_T(X). (5.1)

Here:

S_T(X) = structure extractable from X by an observer bounded by T. (5.2)

H_T(X) = residual content not closed under the same bound. (5.3)

The Residual Governance framework emphasizes that advanced AI architecture should not merely maximize extracted structure; it must also make the remaining residual governable. It also argues that unresolved ambiguity, conflict, fragility, and missing structure should not be treated as rare exceptions, because they are the normal runtime face of bounded observation.

This principle is central to the proposed architecture.

A good AI system should not pretend that residual is gone. It should classify it, carry it, expose it, reduce it where possible, and escalate it when necessary.

Good closure = stable structure + honest residual treatment. (5.4)

Bad closure = fluent structure + hidden residual. (5.5)

5.2 Residual types for domain-specific AI

A DSS system may produce strong specialist answers, but residuals remain. Common residual types include:

- coverage residual: missing relevant documents, cases, tests, or subdomains;
- ambiguity residual: multiple plausible interpretations remain;
- contradiction residual: mature objects conflict;
- boundary residual: the active domain universe may be wrong or incomplete;
- verification residual: the conclusion is not yet checked by the strongest available method;
- action residual: even if the answer is true, its recommended action may be unsafe, costly, or premature;
- novelty residual: the case may represent a new pattern not covered by existing abstractions;
- translation residual: different specialist systems use incompatible terms or schemas.

These residuals should become explicit objects.

ResidualPacket := {kind, source, affected_object, severity, carry_cost, escalation_path}. (5.6)

The important design rule is:

Residual should be governed, not hidden. (5.7)

The PORE layer depends on this. Its baseline judgment must include residual risks. If it does not, it becomes a simplistic executive summary rather than a governance object.

5.3 Specialist disagreement as residual signal

In a DSS society, different specialists may disagree. A legal DSS may advise caution, while an engineering DSS may say the system is technically sound. A medical DSS may propose one diagnosis, while a risk DSS flags a rare but catastrophic alternative. A finance DSS may optimize return, while a compliance DSS flags regulatory exposure.

The system should not resolve such conflict by majority vote alone.

Conflict is not merely noise. It is often a signal that the problem crosses boundaries. (5.8)

This is where the PORE layer becomes useful. It can establish a coarse-grain baseline, then compare specialist deviations against it. If a specialist answer differs from the baseline, the deviation is not automatically accepted or rejected. It becomes a review object.

DeviationResidual := {PORE_Baseline, Expert_Answer, DifferenceType, EvidenceGap, ResidualDelta, BoundaryRisk}. (5.9)

This turns disagreement into a governable artifact.

---

6. The PORE Coarse-Grain Layer

6.1 Definition

The PORE Coarse-Grain Layer is an external governance layer that reads mature knowledge objects, residual packets, coverage ledgers, and specialist summaries, then produces a professional common-sense baseline judgment.

Formally:

PORE_Layer(Q, U) -> PORE_CommonSenseCard. (6.1)

where Q is the query and U is the active universe.

The PORE_CommonSenseCard contains:

PORE_CommonSenseCard := {SituationSummary, Purpose, Object, Residual, Evaluation, BaselineJudgment, FirstInvestigationPath, RedFlags, ConfidenceBand, ExpertMustBeat}. (6.2)

This card is not final truth. It is a baseline judgment artifact.

The card answers:

What would a competent professional, using mature domain common sense, conclude first?

What obvious drivers matter?

What first investigation direction is most natural?

What residuals prevent premature closure?

What must a specialist answer prove if it wants to overturn this baseline?

6.2 The PORE layer as executive common sense

The PORE layer is inspired by a familiar institutional role: the senior reviewer who may not know every technical detail, but can ask the right high-level questions. Good executives, judges, auditors, physicians, engineers, and entrepreneurs often operate at this level. They do not replace specialists, but they prevent specialist analysis from floating away from purpose, scope, risk, and testability.

The PORE layer represents this function in an AI system.

It asks:

What is the point?

What exactly are we talking about?

What remains unresolved?

How will we test whether this judgment is good enough?

This is not anti-intellectual. It is anti-unaccountable complexity.

A specialist answer should be allowed to be complex. But it should not be allowed to be complex without explaining why the complexity earns its keep.

Complexity must pay rent by reducing residual, increasing coverage, or improving action robustness. (6.3)

6.3 Why the PORE layer should be separate

The PORE layer should not be fused too tightly with specialist DSS reasoning. If the same system generates the baseline and the expert answer in one blended pass, the baseline may anchor the specialist analysis, or the expert analysis may contaminate the baseline.

A safer architecture uses three paths:

System A = PORE baseline generator. (6.4)

System B = specialist DSS deep analysis. (6.5)

System C = deviation and superiority adjudicator. (6.6)

This separation creates plural observer paths. The goal is not to multiply agents for theatrical complexity. The goal is to preserve distinct functions:

- baseline judgment;
- specialist reasoning;
- superiority review.

Only after these paths are produced should the system synthesize a final response.

The final answer is therefore not merely an answer. It is the result of governed comparison.

FinalAnswer = Synthesize(PORE_Baseline, SpecialistAnswer, SuperiorityReview, Residuals). (6.7)

---

7. Transition to Part 2

Part 1 established the motivation and foundation:

- monolithic scaling is insufficient for robust domain reasoning;
- DSS offers a strong specialist alternative;
- mature knowledge objects are needed before reliable domain abstraction;
- residual governance is needed to prevent false closure;
- PORE supplies a coarse-grain professional common-sense baseline;
- expert answers must be compared against that baseline.

Part 2 will develop the operational core of the method:

1. contract algebraization of the PORE layer;
2. the PORE CommonSenseCard schema;
3. the Expert Superiority Review protocol;
4. the DeviationResidual object;
5. a full runtime pipeline for PORE-wrapped DSS systems;
6. examples in medicine, law, finance, and engineering.

A Coarse-Grain Governance Layer for Domain-Specific AI: Knowledge Maturation, Residual Control, and Expert Superiority Review

Part 2 — Contract Algebraization, Common-Sense Cards, and Expert Superiority Review

---

8. Contract Algebraization of the PORE Layer

8.1 Why PORE needs algebraization

The PORE layer is intentionally simple. It is a coarse-grain professional judgment grammar, not a full formal reasoning system. Because of that simplicity, it has two opposing risks.

The first risk is under-formalization. If PORE remains only a loose thinking style, then each run may produce a different kind of baseline. One answer may emphasize purpose, another may emphasize risk, another may emphasize action, and another may become a vague executive summary. In that case, the PORE layer cannot serve as a stable comparison surface.

The second risk is over-formalization. If PORE is turned too early into a rigid theorem-proving or ontology system, it may lose its main advantage: rapid executive common-sense judgment across domains.

The correct middle path is contract algebraization.

Contract algebraization means:

PORE is formalized enough to be repeatable, auditable, and comparable, but not so formalized that it stops functioning as a coarse-grain judgment layer. (8.1)

This is different from the stricter abstraction route emphasized by Princeton-style domain-specific AI. In that route, abstractions such as knowledge graphs, ontologies, formal languages, and verification environments become training and reasoning substrates for specialist models. That is appropriate for DSS construction. The PORE layer has a different role. It is not building the specialist reasoning substrate. It is building the judgment surface around the specialist system. Princeton's paper explicitly centers domain-specific superintelligence on high-quality domain data, explicit symbolic abstractions, synthetic curricula, and specialist models; PORE instead wraps the resulting mature systems with a professional baseline and review protocol.

Therefore, PORE algebraization should not begin with proof rules. It should begin with contracts.

8.2 The five strictness dimensions

A useful way to define contract algebraization is:

PORE_Strictness = InputBoundary + OutputSchema + AuthorityLimit + ComparisonSurface + ResidualProtocol. (8.2)

Each term is necessary.

InputBoundary defines what the PORE layer is allowed to read.

OutputSchema defines what the PORE layer must produce.

AuthorityLimit defines what the PORE layer is allowed to decide.

ComparisonSurface defines how expert answers are compared against the baseline.

ResidualProtocol defines what happens when the baseline and specialist answer diverge.

These five dimensions prevent the PORE layer from becoming either a vague essay generator or an overconfident executive oracle.

8.3 Input boundary

The PORE layer should not normally consume raw sources directly. It should consume matured and governed knowledge artifacts.

PORE_Input := {Q, U, K_m, Cov, Res, DSS_sum, Context}. (8.3)

where:

Q = current query or decision problem.

U = active universe or domain perspective.

K_m = mature knowledge objects.

Cov = coverage ledger.

Res = residual packets.

DSS_sum = summaries or outputs from specialist systems.

Context = user, operational, institutional, or decision context.

The reason is architectural. If PORE directly reads raw documents, it becomes another summarizer. If it reads mature knowledge objects, it becomes a judgment compression layer.

The Knowledge Object architecture already makes this distinction. It treats wiki pages not as final knowledge, but as staged objects in a maturation pipeline, moving from raw source-grounded objects toward mature, perspective-bound objects with trace, residual, and coverage governance.

Therefore:

PORE should operate after maturation, not before maturation. (8.4)

There may be exceptions. In a cold-start system with no mature objects, PORE can produce a weak provisional baseline. But the output must clearly mark itself as immature.

ColdStart_PORE = provisional baseline with low coverage confidence. (8.5)

8.4 Output schema

The PORE layer should always produce a fixed artifact. The minimal artifact is the PORE Common-Sense Card.

PORE_CommonSenseCard := {Situation, P, O, R, E, B, F, Z, C, M}. (8.6)

where:

Situation = short statement of the case.

P = Purpose.

O = Object.

R = Residual.

E = Evaluation.

B = Baseline judgment.

F = First investigation path.

Z = red flags.

C = confidence and coverage warning.

M = what expert analysis must beat.

This schema matters because it makes professional common sense inspectable. Without a schema, “common sense” may become a rhetorical posture. With a schema, it becomes a reviewable object.

The card is not a final answer. It is a baseline artifact.

PORE_Card ≠ FinalAnswer. (8.7)

PORE_Card = baseline for governed comparison. (8.8)

8.5 Authority limit

The PORE layer should have limited authority.

It may produce:

- a baseline judgment;
- obvious drivers;
- first investigation direction;
- common red flags;
- residual warnings;
- expert challenge criteria.

It may not produce:

- final specialist verdicts;
- automatic medical, legal, financial, or engineering determinations;
- irreversible action approval;
- hidden updates to mature knowledge;
- unreviewed overrides of specialist systems.

This boundary can be written as:

Authority(PORE) = Baseline + SanityCheck + ChallengeCriteria. (8.9)

Authority(PORE) ≠ SpecialistClosure. (8.10)

This is crucial for safety. The PORE layer can say, “The professional common-sense view suggests X.” It cannot say, “Therefore X is the final truth.”

8.6 Comparison surface

The central purpose of the PORE layer is to create a comparison surface. A specialist answer is not accepted merely because it is more technical. It must show why it is better than the baseline.

Expert_Superiority = Compare(ExpertAnswer, PORE_Baseline). (8.11)

The comparison should not be purely stylistic. It should ask:

- Did the expert use stronger evidence?
- Did it cover more of the relevant domain?
- Did it reduce residual?
- Did it produce a more robust action recommendation?
- Did it introduce unnecessary complexity?
- Did it create new boundary risk?

A first version can be qualitative:

Superiority ∈ {not_shown, weak, moderate, strong}. (8.12)

Later versions may use scores, weights, or domain-specific evaluation functions.

8.7 Residual protocol

If the expert answer confirms the PORE baseline, the residual protocol is simple: record convergence.

If the expert answer refines the baseline, record the refinement.

If the expert answer overturns the baseline, create a DeviationResidual object.

DeviationResidual := {B, A_e, Δ, EvidenceGap, ResidualDelta, ComplexityCost, BoundaryRisk, Escalation}. (8.13)

where:

B = PORE baseline.

A_e = expert answer.

Δ = difference type.

EvidenceGap = what evidence changed the result.

ResidualDelta = what residual is reduced or increased.

ComplexityCost = additional complexity introduced.

BoundaryRisk = risk that the wrong universe or scope is being used.

Escalation = review path if unresolved.

The key governance principle is:

A specialist deviation from common sense should not disappear into prose; it should become a governable object. (8.14)

This turns disagreement into knowledge-system fuel. Over time, repeated deviations can reveal where common sense is too shallow, where expert systems overfit, where the ontology is incomplete, or where the domain itself is changing.

---

9. The PORE Common-Sense Card

9.1 Purpose of the card

The PORE Common-Sense Card is the main artifact of the coarse-grain governance layer. It is designed to be short enough for rapid decision support, but structured enough for expert review and later audit.

The card does three things.

First, it captures the professional common-sense baseline.

Second, it exposes the residuals that prevent false closure.

Third, it defines what a specialist answer must prove if it wants to deviate.

This third function is the most novel. Many AI systems generate a “summary” or “recommendation.” The PORE card generates a baseline plus a challenge surface.

PORE_Card = Baseline + ResidualWarning + ExpertChallenge. (9.1)

9.2 Card fields

A practical PORE Common-Sense Card may contain the following fields.

1. Query ID

A stable identifier for the user query or decision episode.

2. Active Universe

The domain perspective used for the baseline.

Examples:

- medicine;
- law;
- engineering;
- finance;
- compliance;
- operations;
- education;
- product strategy.

3. Situation Summary

A short, neutral restatement of the case.

4. Purpose

The objective or value that controls the judgment.

5. Object

The bounded entity under judgment.

6. Residual

The unresolved uncertainty, ambiguity, conflict, missing information, fragility, or risk.

7. Evaluation

The criterion for judging whether the answer or action is good enough.

8. Baseline Judgment

The first-pass professional common-sense view.

9. Obvious Drivers

The main variables that appear to control the situation.

10. First Investigation Path

The most natural first action or inquiry direction.

11. Red Flags

Conditions that would invalidate the baseline or require escalation.

12. Confidence Band

A qualitative or quantitative confidence estimate.

13. Coverage Warning

A statement about the maturity and coverage of the knowledge base.

14. Expert Must Beat

A list of points that specialist analysis must address if it wants to overturn or materially refine the baseline.

The card can be represented compactly:

$Card_i = \{U_i, S_i, P_i, O_i, R_i, E_i, B_i, D_i, F_i, Z_i, C_i, M_i\}$. (9.2)

where D_i denotes obvious drivers and M_i denotes expert-must-beat conditions.

9.3 The Expert Must Beat field

The most important field is Expert Must Beat.

This field converts the PORE card from a passive summary into a governance instrument.

A weak version says:

Expert must provide stronger evidence if disagreeing. (9.3)

A strong version says:

Expert must specify the missed condition, supporting mature object, residual reduction, action advantage, and added risk. (9.4)

A standard template is:

To beat this baseline, the expert answer must show:

1. what hidden condition the baseline missed;
2. which mature object, evidence source, rule, graph path, or formal check changes the judgment;
3. what residual is reduced by the expert answer;
4. why the baseline action would fail, underperform, or be unsafe;
5. what new complexity, cost, or boundary risk is introduced;
6. why the trade-off is worth accepting.

This makes expert reasoning accountable without suppressing expertise.

9.4 Card maturity levels

Not all PORE cards are equally strong. A card should carry a maturity level.

PORE_Maturity ∈ {cold_start, provisional, covered, mature, audited}. (9.5)

cold_start means the system lacks mature domain objects.

provisional means some mature objects are available but coverage is weak.

covered means the main domain objects are known and coverage is acceptable.

mature means the baseline is grounded in stable mature objects and known residuals.

audited means prior expert reviews and outcomes have tested similar baselines.

The card's authority depends on maturity.

Authority(Card) ∝ Maturity(Card). (9.6)

A cold-start baseline should be treated as a hypothesis. An audited baseline can function as a strong professional prior.

---

10. Expert Superiority Review

10.1 Why superiority review is needed

Specialist AI systems can be impressive and wrong. They can also be right for reasons that are difficult for non-specialists to inspect. A specialist answer may contain citations, equations, graph paths, tool outputs, or formal checks. But the user still needs to know why that expert output improves on the professional baseline.

The Expert Superiority Review is the bridge.

ExpertSuperiorityReview := Compare(PORE_Card, SpecialistAnswer, Evidence, Residuals). (10.1)

It asks whether the specialist answer:

- confirms the baseline;
- refines the baseline;
- overturns the baseline;
- exposes a new residual;
- requires escalation.

The result is not merely "expert says yes/no." It is a structured judgment about the relationship between common sense and expertise.

10.2 The superiority function

A useful semi-formal expression is:

Expert_Superiority = Evidence_Gain + Coverage_Gain + Residual_Reduction + Action_Robustness − Complexity_Cost − Boundary_Risk. (10.2)

Each term can be interpreted qualitatively.

Evidence_Gain asks whether the specialist answer uses stronger or more relevant evidence than the baseline.

Coverage_Gain asks whether it covers more of the relevant object space.

Residual_Reduction asks whether it reduces ambiguity, contradiction, fragility, or missingness.

Action_Robustness asks whether its recommendation works better under realistic constraints.

Complexity_Cost asks whether the answer introduces extra machinery, assumptions, operational burden, or interpretive load.

Boundary_Risk asks whether the specialist answer may be applying the wrong universe, wrong domain frame, wrong scope, or wrong abstraction.

A specialist answer is superior only if the gains exceed the costs.

Accept_Expert if Gains > Costs + ReviewThreshold. (10.3)

where:

Gains = Evidence_Gain + Coverage_Gain + Residual_Reduction + Action_Robustness. (10.4)

Costs = Complexity_Cost + Boundary_Risk. (10.5)

This formula is not meant to create fake precision. It is a disciplined checklist. In early implementations, each component can be rated as low, medium, or high.

10.3 Confirm, refine, overturn, residualize, escalate

The Expert Superiority Review should produce one of five outcomes.

Outcome ∈ {confirm, refine, overturn, residualize, escalate}. (10.6)

confirm means the specialist answer agrees with the PORE baseline and adds no material correction.

refine means the specialist answer improves details but preserves the baseline direction.

overturn means the specialist answer gives sufficient evidence to reject the baseline.

residualize means the specialist answer does not resolve the difference but identifies a meaningful unresolved residual.

escalate means the system should route to a human expert, stronger verification method, additional DSS, or external tool.

These outcomes are important because they prevent binary thinking. Many real cases are not “baseline right” or “expert right.” They are cases where the difference itself becomes a new object of governance.

10.4 Expert review schema

A practical Expert Superiority Review object can be:

ExpertSuperiorityReview := {BaselineID, ExpertAnswerID, Relation, EvidenceGain, CoverageGain, ResidualReduction, ActionRobustness, ComplexityCost, BoundaryRisk, Outcome, Explanation}. (10.7)

where Relation may be:

Relation ∈ {same_direction, narrower, broader, opposite, orthogonal, unresolved}. (10.8)

This allows later auditing. For example, if many expert answers repeatedly overturn PORE baselines in a specific domain, that domain’s PORE pack may be too shallow. If many expert answers add complexity but fail to improve outcomes, the specialist system may be overfitting.

Over time:

ReviewHistory -> PORE_Pack_Update. (10.9)

This creates a learning loop without silently rewriting the baseline.

---

11. Deviation Residuals

11.1 Why deviations should be preserved

When a specialist answer differs from the PORE baseline, the system should not simply choose one. It should preserve the difference as a DeviationResidual unless the superiority review clearly resolves it.

This is because deviations are often high-value signals.

They may indicate:

- the baseline is too crude;
- the specialist model has discovered a hidden variable;
- the specialist model is hallucinating complexity;
- the domain boundary is wrong;
- the mature knowledge layer is incomplete;
- the case is genuinely novel;
- the evaluation criterion is misaligned.

A deviation is not a nuisance. It is a diagnostic event.

Deviation = signal at the boundary between common sense and expertise. (11.1)

11.2 Types of deviation

A useful taxonomy:

1. Baseline-too-shallow deviation

The PORE baseline misses a technical condition that specialists correctly identify.

2. Expert-overfit deviation

The specialist answer adds complexity without improving evidence, coverage, residual reduction, or action robustness.

3. Boundary-conflict deviation

The baseline and specialist answer are operating under different domains or universes.

4. Evidence-asymmetry deviation

The specialist answer has access to evidence not represented in the PORE card.

5. Verification-gap deviation

The specialist answer depends on a claim that should be checked by a stronger verifier.

6. Novel-insight deviation

The specialist answer violates common sense but may represent a real new pattern.

7. Action-risk deviation

The baseline may be directionally right but unsafe or insufficient under real action constraints.

These can be written:

Δ_type ∈ {baseline_shallow, expert_overfit, boundary_conflict, evidence_asymmetry, verification_gap, novel_insight, action_risk}. (11.2)

11.3 Deviation residual object

A full DeviationResidual object:

DeviationResidual := {CardID, ExpertID, Δ_type, ConflictStatement, MissingEvidence, ResidualDelta, VerificationNeed, EscalationPath, LearningValue}. (11.3)

LearningValue is important. A deviation with high learning value should be fed back into the knowledge maturation process.

LearningValue ∈ {low, medium, high, critical}. (11.4)

For example:

- A single expert deviation in a rare medical case may be high learning value.
- A repeated deviation in contract interpretation may indicate the legal PORE pack needs updating.
- A deviation caused by weak source coverage may trigger ingestion of new raw sources.
- A deviation caused by conflicting domain standards may create an Inspirational Object for later assimilation.

Thus:

DeviationResidual -> KnowledgeMaturationQueue. (11.5)

This closes the loop between runtime judgment and long-horizon knowledge improvement.

---

12. Full Runtime Architecture

12.1 Overview

The complete architecture has nine layers.

Layer 1: Raw Sources

Documents, manuals, cases, textbooks, datasets, logs, regulations, scientific papers, interviews, and internal reports.

Layer 2: Raw Objects

Source-grounded extracted objects with provenance.

Layer 3: Mature Objects

Universe-bound consolidated objects with coverage, trace, and residuals.

Layer 4: Symbolic Abstraction Layer

Knowledge graphs, ontologies, formal rules, proof environments, simulation interfaces, program libraries, and structured constraints.

Layer 5: Curriculum and DSS Training Layer

Synthetic reasoning tasks, multi-hop questions, proof traces, verifier-grounded examples, and domain-specific fine-tuning or RL signals.

Layer 6: DSS Runtime Society

Specialist SLMs, tools, retrievers, verifiers, and domain-specific agents.

Layer 7: Residual-Governed Coordination Runtime

Skill cells, artifact contracts, coordination episodes, trace logs, residual packets, and escalation routes.

Layer 8: PORE Coarse-Grain Judgment Layer

Professional common-sense baseline generation.

Layer 9: Expert Superiority Review

Comparison between PORE baseline and specialist outputs, producing confirm/refine/overturn/residualize/escalate outcomes.

The full pipeline:

RawSources -> RawObjects -> MatureObjects -> Abstractions -> Curricula -> DSS -> ResidualRuntime -> PORE_Baseline -> ExpertReview -> FinalAnswer. (12.1)

12.2 Separation of observer paths

A safe implementation separates three paths.

Path A: PORE baseline path.

Path B: Specialist DSS path.

Path C: Adjudication path.

Path A:

$A_PORE = C_PORE(K_m, Cov, Res, Q, U)$. (12.2)

Path B:

$A_DSS = SpecialistReason(Q, U, K_m, Tools, Abstractions)$. (12.3)

Path C:

$A_Final = Adjudicate(A_PORE, A_DSS, Res, Cov)$. (12.4)

The reason for separation is to reduce premature contamination. If the PORE baseline sees the specialist answer first, it may rationalize it. If the specialist system sees the baseline too strongly, it may anchor on it. If the adjudicator is not separate, the system may hide disagreement.

This is not a call for unnecessary multi-agent theater. It is a call for functional separation.

Functional separation is justified when the outputs must challenge each other. (12.5)

12.3 Skill cells inside the runtime

The DSS society should not be implemented merely as a list of named agents. “Legal Agent,” “Medical Agent,” “Engineering Agent,” and “Finance Agent” are useful labels, but they are too broad for reliable runtime governance.

Instead, each specialist system should be decomposed into skill cells.

SkillCell := {Scope, EntryCondition, InputContract, Transformation, OutputContract, ExitCondition, FailureModes}. (12.6)

Examples:

- QueryClarificationCell;
- EvidenceRetrievalCell;
- KGPathValidationCell;
- ContradictionDetectionCell;
- FormalCheckCell;
- RiskEscalationCell;
- HumanReviewPackagingCell;
- PORECardGenerationCell;
- ExpertSuperiorityReviewCell.

This is consistent with the Residual Governance view that vague role names should be replaced by bounded transformation units, coordination episodes, maintained structure, artifact contracts, and traceable residuals.

The PORE layer itself can be implemented as a small set of cells:

PORE_SituationCell. (12.7)

PORE_PurposeCell. (12.8)

PORE_ObjectCell. (12.9)

PORE_ResidualCell. (12.10)

PORE_EvaluationCell. (12.11)

PORE_BaselineCell. (12.12)

PORE_ChallengeCell. (12.13)

Together:

PORE_Card = Compose(PORE_SituationCell, PORE_PurposeCell, PORE_ObjectCell, PORE_ResidualCell, PORE_EvaluationCell, PORE_BaselineCell, PORE_ChallengeCell). (12.14)

---

13. Examples

13.1 Medical example

Suppose the query is:

A patient has persistent chest discomfort, borderline ECG changes, and a history of anxiety. What should be done next?

A PORE Common-Sense Card might say:

Purpose: avoid missing life-threatening cardiac events while preventing unnecessary intervention.

Object: the patient's current symptom episode, risk profile, ECG pattern, and available clinical history.

Residual: incomplete cardiac enzymes, unclear onset timing, possible atypical presentation, anxiety as confounder but not sufficient exclusion.

Evaluation: safety of triage decision, guideline consistency, need for urgent testing, acceptable false-negative risk.

Baseline Judgment: do not dismiss as anxiety without ruling out cardiac risk; obtain appropriate urgent evaluation.

First Investigation Path: repeat ECG, troponin testing, risk stratification, review medication and history.

Expert Must Beat: any specialist answer proposing low-risk discharge must explain why cardiac residual is acceptably reduced.

A cardiology DSS may then refine the baseline with guideline-based stratification. If it says discharge is safe, it must prove residual reduction through evidence, not reassurance.

This is the function of PORE: it makes the obvious safety baseline explicit, then forces specialist deviation to justify itself.

13.2 Legal example

Query:

A company wants to terminate a supplier contract after repeated delays. What is the likely legal risk?

PORE baseline:

Purpose: protect commercial interests while minimizing breach-of-contract exposure.

Object: supplier contract, delay events, notice history, force majeure clauses, termination provisions, jurisdiction.

Residual: unclear whether delays are material breach, whether notice requirements were satisfied, whether the supplier has defenses.

Evaluation: enforceability of termination, litigation exposure, damages, negotiation leverage.

Baseline Judgment: do not terminate solely on frustration; first verify contractual notice, cure periods, documented breach, and available remedies.

Expert Must Beat: any aggressive termination recommendation must show controlling clauses, evidence of material breach, and reduced litigation residual.

A legal DSS may later say immediate termination is justified. But it must show why the cautious common-sense baseline is too conservative.

13.3 Finance example

Query:

A company's revenue is growing quickly, but cash flow is deteriorating. Is the business healthy?

PORE baseline:

Purpose: assess sustainable business quality, not just top-line growth.

Object: revenue growth, cash conversion cycle, receivables, inventory, working capital, margin, debt, customer concentration.

Residual: unclear whether growth is profitable, financed by delayed collection, or driven by low-quality revenue.

Evaluation: cash conversion, operating margin, free cash flow, receivable aging, debt service capacity.

Baseline Judgment: revenue growth alone is not sufficient; deteriorating cash flow is a red flag requiring working-capital analysis.

Expert Must Beat: any optimistic answer must show that cash deterioration is temporary, explained, and reversible.

A finance DSS may confirm or refine. If it claims the company is healthy, it must provide the evidence gain.

13.4 Engineering example

Query:

A system works in testing but fails intermittently in production. What should be investigated first?

PORE baseline:

Purpose: restore reliability and identify production-specific failure drivers.

Object: the production system, deployment environment, traffic pattern, dependencies, logs, configuration differences.

Residual: incomplete reproduction, hidden load differences, race conditions, environment mismatch, observability gaps.

Evaluation: reproducibility, failure frequency, blast radius, rollback safety, root-cause confidence.

Baseline Judgment: prioritize environment/configuration differences, load-related behavior, dependency instability, and logging gaps before rewriting core logic.

Expert Must Beat: any deep algorithmic explanation must show why simpler production-environment causes are insufficient.

A specialist engineering DSS may identify a subtle concurrency bug. If so, it must show reproduction evidence or trace-level support.

The PORE layer prevents premature fascination with sophisticated explanations.

---

14. Transition to Part 3

Part 2 developed the operational core:

- PORE should be algebraized as a contract, not as a theorem prover;
- the PORE Common-Sense Card makes professional judgment inspectable;
- Expert Superiority Review forces specialist answers to prove their value;
- DeviationResiduals turn disagreement into governance material;
- the full architecture separates knowledge maturation, DSS reasoning, residual control, PORE baseline, and adjudication;
- examples show how the method works across medicine, law, finance, and engineering.

Part 3 will complete the paper by developing:

1. evaluation protocols;
2. implementation ladder;
3. domain PORE packs;
4. failure modes and safeguards;
5. relationship to DSS, RAG, agents, and knowledge graphs;
6. organizational implications;
7. conclusion and appendices.

A Coarse-Grain Governance Layer for Domain-Specific AI: Knowledge Maturation, Residual Control, and Expert Superiority Review

Part 3 — Evaluation, Implementation, Failure Modes, and Organizational Use

---

15. Evaluation Protocols

15.1 Why this architecture must be evaluated differently

A PORE-wrapped domain-specific AI system should not be evaluated only by answer accuracy. Accuracy remains important, but it is not sufficient. The proposed architecture makes a stronger claim: it should improve the quality of governed judgment. That means evaluation must ask whether the system produces answers that are not only correct, but also better framed, more auditable, more residual-aware, and more justifiably superior to a professional common-sense baseline.

Traditional LLM evaluation often asks:

Did the model give the right answer? (15.1)

A DSS evaluation may ask:

Did the domain specialist solve the domain task better than a generalist? (15.2)

A PORE-wrapped governance evaluation must ask:

Did the system show why the specialist answer is better than the disciplined professional baseline? (15.3)

This is a different evaluation target. It is not merely answer quality. It is judgment quality under structured comparison.

The Princeton DSS proposal already shifts attention from monolithic breadth toward domain-specific depth, high-quality data, explicit abstraction, and modular specialist societies. It argues that current scale-driven models face limits in reasoning depth, data quality, training and inference costs, and that specialist models grounded in explicit abstractions can achieve deeper reasoning with smaller footprints. The present framework accepts that shift but adds another evaluation demand: the expert system must show its superiority relative to coarse-grain professional common sense.

15.2 Four evaluation baselines

A useful experiment should compare at least four system variants.

Baseline A: Direct Generalist Answer. (15.4)

Baseline B: Specialist DSS Answer. (15.5)

Baseline C: PORE Baseline Only. (15.6)

Baseline D: PORE-Wrapped DSS Answer. (15.7)

Baseline A measures what a general-purpose LLM would say.

Baseline B measures what a specialist domain system would say.

Baseline C measures what the PORE layer says without specialist deep reasoning.

Baseline D measures the full system: PORE baseline, specialist answer, expert superiority review, residual disclosure, and final synthesis.

The key comparison is not only whether D is more correct than B. The deeper question is whether D is more governable than B.

Governance_Gain = Quality(D) − Quality(B). (15.8)

where Quality includes not only correctness, but also clarity, residual honesty, justification, auditability, and action robustness.

15.3 Core metrics

The following metrics can be used.

1. Accuracy or task success

This is the ordinary domain metric.

Examples:

- correct diagnosis;
- correct legal issue identification;
- correct engineering fault classification;
- correct financial risk assessment;
- correct code repair;
- correct retrieval of controlling documents.

Accuracy remains necessary.

Accuracy = correct_outputs / total_outputs. (15.9)

But it is not enough.

---

2. Baseline clarity

Does the PORE Common-Sense Card state the first-pass professional view clearly?

Baseline_Clarity ∈ {poor, acceptable, strong}. (15.10)

A strong baseline should identify:

- the purpose;
- the object;
- the main residual;
- the evaluation criterion;
- the natural first investigation path;
- what experts must beat.

If the baseline is vague, the later expert comparison becomes weak.

---

3. Expert superiority quality

Does the specialist answer explain why it improves on the PORE baseline?

Expert_Superiority_Quality = f(EvidenceGain, CoverageGain, ResidualReduction, ActionRobustness, ComplexityCost, BoundaryRisk). (15.11)

A high-quality expert superiority review should state:

- what the baseline got right;
- what the baseline missed;
- what evidence changes the decision;
- what residual is reduced;
- what complexity is introduced;
- why the deviation is justified.

---

4. Residual honesty

Does the system preserve unresolved ambiguity instead of smoothing it into false certainty?

Residual_Honesty = visible_residual / material_residual. (15.12)

This is hard to measure directly, but it can be approximated by expert audit. Human reviewers can ask:

- Did the system disclose important uncertainty?
- Did it hide conflicting evidence?
- Did it overstate coverage?
- Did it preserve meaningful rival interpretations?
- Did it escalate when needed?

The Residual Governance framework treats unresolved ambiguity, contradiction, fragility, bridge failure, and boundary leakage as normal runtime residuals rather than edge cases. It also stresses that systems need better units, clocks, states, traces, and escalation discipline once false closure becomes costly.

---

5. Action robustness

Does the final recommendation remain reasonable under realistic perturbations?

Action_Robustness = stability_of_recommendation_under_boundary_conditions. (15.13)

Examples:

- If one evidence source is removed, does the recommendation collapse?
- If the user's domain is slightly different, does the result still apply?
- If a hidden assumption fails, does the action become dangerous?
- If cost, time, or legal constraints change, is the answer still usable?

This matters because professional judgment is not merely about truth. It is about safe action under imperfect closure.

---

6. Complexity discipline

Does the specialist answer introduce complexity that actually improves the result?

Complexity_Discipline = useful_complexity / total_complexity. (15.14)

A specialist answer may cite many details, but only some details matter. The PORE layer helps test whether extra complexity pays rent.

Complexity pays rent only if it increases evidence, coverage, residual reduction, or action robustness. (15.15)

---

7. Audit replayability

Can the system later reconstruct why it answered as it did?

Replayability = recoverable_decision_trace / required_decision_trace. (15.16)

A replayable output should preserve:

- PORE card ID;
- mature objects consulted;
- specialist systems activated;
- artifact contracts used;
- residual packets carried;
- superiority review result;
- final synthesis rationale.

Without replayability, governance becomes performance theater.

15.4 Evaluation matrix

A simple evaluation matrix may look like this:

Evaluation Dimension	Direct LLM	DSS Only	PORE Only	PORE-Wrapped DSS
Domain accuracy	medium	high	medium	high
Baseline clarity	low	low/medium	high	high
Expert superiority explanation	low	medium	none	high
Residual honesty	variable	medium	medium	high
Action robustness	variable	high if DSS is good	medium	high
Audit replayability	low	medium	medium	high
Complexity discipline	low	variable	high	high

The hypothesis of this paper is:

PORE-Wrapped DSS should outperform DSS-only systems on governed judgment metrics, even when raw accuracy is similar. (15.17)

---

16. Implementation Ladder

16.1 Do not build the full stack first

The proposed architecture can become large. That is a danger. A sensible system should not begin with every layer. It should begin with the smallest version that improves control, trust, and decision quality enough to justify its cost.

The Knowledge Object framework makes a similar point: it is not a mandatory total architecture, but a selectable maturation framework; additional machinery should be inserted only when the current layer can no longer govern staleness, drift, or dishonesty honestly.

The same principle applies here.

Add the next governance layer only when it reduces drift, false closure, re-derivation, or expert opacity enough to justify its cost. (16.1)

16.2 Stage 1: PORE card as structured prompt output

The minimal implementation is simple.

Input:

- user query;
- active domain;
- short context;
- available mature knowledge summary.

Output:

- PORE Common-Sense Card.

At this stage, the system may not yet have full knowledge objects, residual packets, or mature DSS backends. It can still enforce a basic discipline:

PORE_Minimal(Q, U) -> {P, O, R, E, Baseline, ExpertMustBeat}. (16.2)

This already improves many AI answers because it prevents immediate expert-style prose without first establishing purpose, object, residual, and evaluation.

Use case:

- personal research assistant;
- lightweight consulting tool;
- early enterprise prototype;
- document review assistant.

The risk is that the card may be only superficially grounded. Therefore, it must carry a maturity label.

PORE_Maturity = cold_start or provisional. (16.3)

16.3 Stage 2: PORE card connected to mature knowledge objects

The second stage connects the PORE layer to a governed knowledge base.

Input now includes:

- mature objects;
- coverage ledger;
- known residual packets;
- domain universe;
- prior reviewed cases.

PORE_KO(Q, U, K_m, Cov, Res) -> PORE_CommonSenseCard. (16.4)

This is the first serious version. The PORE baseline is no longer just an LLM-generated professional guess. It becomes a coarse-grain projection of a mature domain knowledge layer.

At this stage, the system should store:

- which mature objects contributed to Purpose;
- which contributed to Object definition;
- which residual packets shaped Residual;
- which evaluation criteria were selected;
- what coverage warning applies.

The output becomes replayable.

16.4 Stage 3: Add specialist DSS and expert superiority review

The third stage adds specialist reasoning.

DSS_Answer = SpecialistReason(Q, U, K_m, Tools, Abstractions). (16.5)

Review = Compare(PORE_Card, DSS_Answer). (16.6)

Final = Synthesize(PORE_Card, DSS_Answer, Review, Residuals). (16.7)

This is the first full PORE-wrapped DSS runtime.

At this stage, the key operational question becomes:

Does the specialist answer confirm, refine, overturn, residualize, or escalate the baseline? (16.8)

The system should not merely combine answers. It should classify their relation.

16.5 Stage 4: Add deviation residual learning loop

The fourth stage stores unresolved deviations.

DeviationResidual -> MaturationQueue. (16.9)

This allows the system to improve over time.

Repeated deviations may indicate:

- PORE baseline is too shallow;
- a domain PORE pack needs revision;

- mature objects are under-covered;
- the DSS abstraction layer is missing relations;
- the specialist model is overfitting;
- a new domain subcategory is emerging.

At this stage, the architecture becomes a learning governance system. It does not silently self-edit truth. It accumulates reviewable residuals that can later be assimilated.

16.6 Stage 5: Domain PORE packs

The fifth stage creates domain-specific PORE packs.

Domain_PORE_Pack(U) := {DriverSet, RedFlagSet, EvaluationSet, ResidualTaxonomy, EvidenceHierarchy, FirstInvestigationPatterns}. (16.10)

Examples:

Finance_PORE_Pack. (16.11)

Medical_PORE_Pack. (16.12)

Legal_PORE_Pack. (16.13)

Engineering_PORE_Pack. (16.14)

Each pack adapts PORE to a domain without changing the PORE kernel.

This mirrors the Universe Pack idea in Knowledge Object architecture, where a universe pack may define object classes, segmentation rules, merge admissibility rules, residual taxonomy, indexing strategy, and evaluation criteria.

The PORE kernel remains:

PORE = Purpose + Object + Residual + Evaluation. (16.15)

Only the domain interpretation changes.

---

17. Domain PORE Packs

17.1 Why domain packs are needed

A generic PORE card is useful, but domain judgment depends on domain-specific drivers and evaluation standards. The medical meaning of “residual” differs from the financial meaning. The legal meaning of “object” differs from the engineering meaning. The same four coordinates remain, but their admissible content changes.

Therefore, each domain can define a PORE pack.

Pack_PORE(U) = {P_rules, O_rules, R_rules, E_rules, DriverMap, RedFlags, EvidenceRank}. (17.1)

where:

P_rules define common purposes in that domain.

O_rules define object boundaries.

R_rules define residual taxonomy.

E_rules define evaluation criteria.

DriverMap defines obvious causal or operational drivers.

RedFlags define escalation conditions.

EvidenceRank defines what counts as stronger evidence.

17.2 Medical PORE pack

A medical PORE pack may include:

Purpose rules:

- protect patient safety;
- avoid missing severe disease;
- minimize harm from unnecessary intervention;
- preserve informed clinical judgment.

Object rules:

- patient episode;
- symptom cluster;
- diagnosis candidate;
- treatment decision;
- lab or imaging finding;
- guideline applicability.

Residual taxonomy:

- missing test;
- atypical presentation;
- differential diagnosis uncertainty;
- rare-but-serious condition;
- medication interaction;
- incomplete history;
- guideline mismatch.

Evaluation criteria:

- clinical safety;
- guideline consistency;
- diagnostic confidence;
- need for escalation;
- harm-benefit balance.

Red flags:

- unstable vital signs;
- high-risk symptoms;

- conflicting lab results;
- insufficient history;
- irreversible treatment decision.

Medical_PORE = Safety-first baseline under incomplete clinical closure. (17.2)

17.3 Legal PORE pack

A legal PORE pack may include:

Purpose rules:

- resolve dispute under controlling authority;
- preserve procedural rights;
- reduce litigation or compliance exposure;
- protect client interests within lawful bounds.

Object rules:

- claim;
- fact pattern;
- jurisdiction;
- statute;
- contract clause;
- precedent;
- regulatory obligation.

Residual taxonomy:

- disputed fact;
- missing evidence;
- unclear jurisdiction;
- conflicting precedent;
- interpretive ambiguity;
- procedural defect;
- enforcement risk.

Evaluation criteria:

- controlling authority;
- burden of proof;
- remedy viability;
- procedural compliance;
- litigation risk;
- settlement leverage.

Red flags:

- fabricated citation;
- missing jurisdiction;
- expired limitation period;
- ambiguous contractual notice;
- unsupported factual assumption.

Legal_PORE = Authority-and-risk baseline under contested facts. (17.3)

17.4 Finance PORE pack

A finance PORE pack may include:

Purpose rules:

- assess economic value;
- protect liquidity;
- manage risk-adjusted return;
- preserve compliance;
- support capital allocation.

Object rules:

- company;
- investment;
- transaction;
- instrument;
- financial statement line;
- risk exposure;
- cash flow pattern.

Residual taxonomy:

- accounting uncertainty;
- valuation sensitivity;
- cash-flow mismatch;
- liquidity risk;
- leverage risk;
- concentration exposure;

- regulatory uncertainty;
- macro regime risk.

Evaluation criteria:

- free cash flow;
- return on capital;
- downside protection;
- liquidity runway;
- covenant compliance;
- risk-adjusted return.

Red flags:

- revenue growth without cash conversion;
- margin expansion without operating evidence;
- opaque related-party transactions;
- debt maturity concentration;
- aggressive accounting assumptions.

Finance_PORE = Cash-and-risk baseline under economic uncertainty. (17.4)

17.5 Engineering PORE pack

An engineering PORE pack may include:

Purpose rules:

- ensure system function;
- preserve safety;
- improve reliability;
- reduce failure rate;
- control cost and maintainability.

Object rules:

- component;
- interface;
- failure mode;
- deployment environment;
- test result;
- dependency;
- configuration;
- operational workflow.

Residual taxonomy:

- unreproduced failure;
- missing telemetry;
- environment mismatch;
- hidden dependency;
- race condition;
- load sensitivity;
- safety margin uncertainty;
- user-behavior mismatch.

Evaluation criteria:

- reproducibility;
- test coverage;
- failure frequency;
- blast radius;
- mean time to repair;
- safety margin;
- maintainability.

Red flags:

- works in test but fails in production;
- no logs around failure;
- unexplained intermittent behavior;
- configuration drift;
- single point of failure;
- unbounded retry loop;
- untested rollback.

Engineering_PORE = Reliability-and-reproduction baseline under system uncertainty. (17.5)

---

18. Failure Modes and Safeguards

18.1 Failure mode: PORE becomes shallow managerial prose

The PORE layer may degenerate into vague leadership language.

Symptoms:

- “focus on the objective” without specifying the objective;

- “manage risk” without identifying residual;
- “evaluate carefully” without naming evaluation criteria;
- “experts should review” without challenge conditions.

Safeguard:

Reject PORE cards that do not fill P, O, R, and E with domain-specific content.

Invalid_PORE if empty(P) or empty(O) or empty(R) or empty(E). (18.1)

A PORE card is not a motivational memo. It is a judgment object.

18.2 Failure mode: PORE overrules experts too strongly

The PORE baseline may become an anchor that suppresses specialist insight.

Symptoms:

- specialist deviations are treated as suspicious by default;
- rare cases are forced into common patterns;
- novel insights are rejected because they violate common sense;
- the system prefers simple answers even when the evidence supports complexity.

Safeguard:

The PORE layer has authority limits.

Authority(PORE) ≠ FinalVerdict. (18.2)

Specialists can overturn the baseline if they provide sufficient evidence gain, residual reduction, and action robustness.

This protects expertise.

18.3 Failure mode: specialist complexity overwhelms the baseline

The opposite failure occurs when expert systems bury the baseline under technical detail.

Symptoms:

- long specialist explanations without clear superiority;
- many citations but no comparison;
- complex graph path without action consequence;
- formal verification of a narrow point while broader residual remains untouched.

Safeguard:

Require expert superiority review.

Expert_Answer must answer ExpertMustBeat fields. (18.3)

If not, the answer is not rejected automatically, but it is marked:

Superiority_Status = not_shown. (18.4)

18.4 Failure mode: fake algebra

The system may assign numerical scores to evidence gain, residual reduction, or action robustness without real calibration.

Symptoms:

- arbitrary weighted scores;
- false precision;
- unexplained confidence values;
- numerical authority masking subjective judgment.

Safeguard:

Start with qualitative categories.

Score ∈ {low, medium, high, unknown}. (18.5)

Use numeric scores only after calibration with domain outcomes.

No calibration, no precision. (18.6)

18.5 Failure mode: residual overload

If every uncertainty becomes a residual packet, the system may become unusable.

Symptoms:

- too many residual objects;
- no prioritization;
- users overwhelmed by caveats;
- system refuses to close.

Safeguard:

Classify residual by action relevance.

ResidualPriority = severity × likelihood × action_relevance. (18.7)

Only high-priority residuals should block closure. Lower-priority residuals can be carried in background.

Closure does not require zero residual. It requires governable residual. (18.8)

18.6 Failure mode: domain boundary error

The PORE card may use the wrong active universe.

Example:

- treating a legal-risk question as a finance question;
- treating a medical triage issue as a customer-service issue;
- treating an engineering safety issue as a product-experience issue.

Safeguard:

Require active universe declaration.

Every PORE card must state U. (18.9)

If multiple universes are plausible, create parallel PORE cards.

PORE_parallel = {Card(U_1), Card(U_2), ..., Card(U_n)}. (18.10)

Then compare them.

This avoids hidden perspective mixing.

18.7 Failure mode: premature maturation

The system may treat immature knowledge as mature and generate confident PORE baselines from weak material.

Safeguard:

Tie PORE authority to knowledge maturity.

Authority(Card) ∝ KnowledgeMaturity × CoverageQuality. (18.11)

If maturity is low, the baseline must be marked provisional.

The Knowledge Object architecture makes this caution central: raw objects are not final doctrine; they preserve source-local extraction for later universe-bound assimilation, while mature objects require explicit perspective discipline, trace, residual, and coverage governance.

---

19. Relationship to Existing AI Patterns

19.1 Relation to RAG

RAG retrieves relevant documents and injects them into generation.

PORE-wrapped governance asks a different question:

What professional baseline follows from mature knowledge, and how must expert reasoning beat it? (19.1)

RAG may be one component of the system, especially in the specialist path. But PORE is not retrieval. It is judgment compression and comparison governance.

RAG retrieves. PORE frames. DSS reasons. Review adjudicates. (19.2)

19.2 Relation to knowledge graphs

Knowledge graphs represent entities and relations. They can support retrieval, reasoning paths, synthetic curricula, and verification. In Princeton's DSS trajectory, KGs and related abstractions are central because they provide explicit structure from which specialist models can learn and reason.

PORE does not replace KGs.

Instead:

KG = structured domain relation substrate. (19.3)

PORE = coarse-grain judgment envelope. (19.4)

A KG may tell the system that A relates to B through relation r. PORE asks why that relation matters for the current purpose, what object is being judged, what residual remains, and how the answer should be evaluated.

In other words:

KG supports domain reasoning. PORE supports judgment governance. (19.5)

19.3 Relation to agents

Agent frameworks coordinate models, tools, memory, and actions. They often use planners, routers, critics, verifiers, and tool callers. The risk is that the architecture becomes a collection of role names without clear runtime objects.

Residual Governance warns against this exact problem: adding planners, verifiers, critics, summarizers, retrieval judges, and tool routers can improve surface vocabulary while reducing runtime legibility if bounded processes, artifact contracts, and residual states are not explicit.

PORE-wrapped governance does not reject agents. It requires agents to produce typed artifacts:

- PORE cards;
- specialist answers;
- superiority reviews;
- deviation residuals;
- escalation packets.

Thus:

Agent output should become a governed artifact, not merely another message. (19.6)

19.4 Relation to formal methods

Formal methods, proof assistants, compilers, solvers, and validators are powerful where the domain permits strict checking. They belong mainly inside the specialist DSS path.

PORE is less strict but more general.

Formal_Check(x) -> pass/fail/proof. (19.7)

PORE_Check(x) -> purpose/object/residual/evaluation/baseline. (19.8)

The two are complementary. A formal proof may establish a claim, but PORE may still ask whether that claim is the right object, whether the purpose is served, and whether remaining residuals matter for action.

19.5 Relation to executive dashboards

Executive dashboards summarize metrics. PORE cards differ in that they are not only descriptive. They are normative judgment artifacts.

A dashboard may say:

Revenue up, cash flow down. (19.9)

A PORE card says:

Purpose: assess sustainable business quality.

Object: revenue growth and cash conversion.

Residual: unclear whether growth is high quality.

Evaluation: free cash flow, working capital, margin, debt service.

Baseline: cash deterioration is a red flag; analyze working capital before accepting growth narrative. (19.10)

The PORE card turns metrics into governed judgment.

---

20. Organizational Implications

20.1 AI as a professional institution, not just a tool

If AI systems become domain-specific societies with mature knowledge, residual governance, and PORE baselines, they begin to resemble professional institutions rather than simple software tools.

A professional institution has:

- knowledge archives;
- expert roles;
- review procedures;
- escalation paths;
- audit trails;
- common-sense norms;
- mechanisms for updating doctrine.

A mature AI runtime needs analogous structures.

AI_Runtime = Knowledge + Experts + Residuals + Review + Trace + Judgment. (20.1)

The PORE layer supplies the common-sense judgment surface in this institution.

20.2 Human experts become judges of superiority

In this architecture, human experts are not merely data labelers or final approvers. They become judges of superiority.

Their role is to evaluate:

- whether the PORE baseline is reasonable;
- whether the specialist answer truly improves it;
- whether residuals are correctly classified;
- whether domain packs need revision;
- whether deviation residuals indicate new doctrine.

This is a higher-value human role.

Human_Expert = arbiter of baseline, deviation, and maturation. (20.2)

This is different from asking humans only to rate answers.

20.3 Decision meetings become more structured

A PORE-wrapped AI system could change how organizations run decision meetings.

Instead of asking:

What does the AI recommend? (20.3)

the meeting asks:

What is the PORE baseline?

What do the specialists say?

Where do they differ?

Has the expert answer proven superiority?

What residual remains?

What decision can close safely?

This mirrors good institutional thinking. The obvious view is not ignored. The expert view is not blindly accepted. Their difference becomes the object of governance.

20.4 Training people to use the system

Users must be trained not only to read final answers, but to read the relation among artifacts.

A good user should understand:

- the PORE baseline is not final truth;
- specialist complexity is not automatically superior;
- residuals are not failures;
- escalation is sometimes the correct outcome;
- a strong answer explains its superiority.

This suggests a new AI literacy requirement.

AI literacy should include baseline reasoning, residual reading, and expert-superiority judgment. (20.4)

---

21. Research Questions

The framework raises several research questions.

21.1 Baseline quality

How do we measure whether a PORE baseline is a good professional common-sense baseline?

Possible methods:

- expert rating;
- outcome comparison;
- baseline stability across equivalent cases;
- sensitivity to coverage warnings;
- ability to identify correct first investigation paths.

21.2 Superiority review calibration

How should Evidence_Gain, Coverage_Gain, Residual_Reduction, Action_Robustness, Complexity_Cost, and Boundary_Risk be calibrated?

Possible approaches:

- qualitative expert labels;

- outcome-based scoring;
- domain-specific rubrics;
- retrospective case review;
- simulation tasks.

21.3 Domain pack learning

How should domain PORE packs evolve?

Possible triggers:

- repeated expert overturns;
- repeated baseline failures;
- repeated residual overload;
- new regulations or scientific findings;
- emergent failure modes.

A conservative update rule:

Update(Pack_U) only after reviewed deviation cluster exceeds threshold. (21.1)

21.4 Multi-universe conflict

How should the system handle cases where multiple universes generate different valid baselines?

For example:

- finance says proceed;
- legal says pause;
- engineering says more testing needed;
- public relations says disclose early.

This requires cross-universe arbitration.

Arbitration = Compare(Card(U_1), Card(U_2), ..., Card(U_n), DecisionContext). (21.2)

The system should not prematurely collapse all universes into one answer.

21.5 Human-AI governance

When should human review be mandatory?

Possible triggers:

- high action irreversibility;
- high residual severity;
- unresolved domain conflict;
- low knowledge maturity;
- specialist answer overturns mature baseline;
- legal, medical, safety, or ethical risk.

HumanReviewRequired if Risk × Irreversibility × ResidualSeverity > Threshold. (21.3)

---

22. Conclusion

The argument of this paper can now be stated compactly.

The future of reliable AI should not be understood as a choice between one giant generalist model and a set of narrow specialist tools. A more promising direction is a layered governed judgment runtime.

At the bottom, raw sources must be converted into knowledge objects. Those objects must mature under declared domain perspectives, with coverage, trace, and residual governance. On top of that mature knowledge, domain-specific specialist systems can be trained and deployed. These systems may use knowledge graphs, ontologies, formal languages, simulators, verification tools, and synthetic curricula to produce deep domain reasoning. But even this is not enough.

Specialist reasoning needs a judgment surface.

The PORE coarse-grain layer supplies that surface. It compresses mature domain knowledge into a professional common-sense baseline: Purpose, Object, Residual, and Evaluation. It then asks every expert answer to confirm, refine, or outperform that baseline.

The central rule is:

A specialist answer is not superior because it is more technical; it is superior only if it shows evidence gain, coverage gain, residual reduction, or action robustness that outweighs its added complexity and boundary risk. (22.1)

This rule protects both sides. It protects common sense from being dismissed by jargon. It protects expertise from being suppressed by shallow intuition. Most importantly, it turns the difference between common sense and expertise into a governable object.

The resulting architecture can be summarized as:

Raw Sources -> Mature Knowledge Objects -> DSS Specialists -> Residual-Governed Runtime -> PORE Baseline -> Expert Superiority Review -> Governed Judgment. (22.2)

This is the paper's final claim:

The next stage of domain-specific AI should not merely be expert AI. It should be common-sense-governed expert AI: a system in which specialist intelligence must continuously prove its superiority over disciplined professional common sense, while unresolved residuals remain visible, typed, and governable. (22.3)

---

23. Operational Capsule: The Six-Step Governed Judgment Loop

For practical implementation, the whole framework can be reduced to one repeatable operational loop. This loop is useful because it shows how the architecture moves from raw domain knowledge to a governed answer, without collapsing expert reasoning, common sense, and residual uncertainty into one opaque response.

The method can be stated as six steps.

Step 1: Mature the knowledge before using it

Before the system answers the user, it should avoid relying directly on raw documents, loose notes, or isolated retrieved chunks. The first task is to convert raw sources into governed knowledge objects.

This means moving from:

RawSources → RawObjects → MatureKnowledgeObjects. (23.1)

A Mature Knowledge Object should carry at least four things: its domain or active universe, its supporting evidence, its boundary conditions, and its known residuals. This step corresponds to the knowledge maturation process discussed in Sections 4 and 16.

In implementation terms, the system retrieves or constructs:

K_m = mature knowledge objects relevant to query Q under universe U. (23.2)

Step 2: Generate the PORE professional common-sense baseline

After mature knowledge has been retrieved, the PORE layer produces a coarse-grain professional baseline. This is not the final expert answer. It is the disciplined first-pass judgment that a competent professional might form before deeper specialist analysis.

The PORE card should identify:

Purpose: what the judgment is trying to serve.

Object: what bounded case, system, claim, or decision is being judged.

Residual: what remains unresolved or risky.

Evaluation: what test or criterion should determine whether the answer is good enough.

In compact form:

PORE_Baseline = PORE(K_m, Q, U, Res, Cov). (23.3)

Here, Res means known residual packets, and Cov means the coverage ledger. This step corresponds to Sections 6, 8, and 9.

Step 3: Run specialist DSS reasoning

Next, the domain-specific specialist system performs deeper reasoning. This may involve a specialist small language model, knowledge graph traversal, formal verification, retrieval tools, simulation, expert rules, or a multi-agent DSS society.

The specialist system is allowed to confirm, refine, or overturn the PORE baseline. However, if it differs from the baseline, it must explain why.

DSS_Answer = DSS(Q, U, K_m, Tools, Abstractions). (23.4)

This step corresponds to Sections 2, 10, 12, and 19. The purpose is not to make expert reasoning subordinate to common sense, but to require expert reasoning to become accountable to a visible baseline.

Step 4: Compare the specialist answer against the baseline

The Expert Superiority Review compares the specialist answer with the PORE baseline. The review should not merely ask whether the specialist sounds more technical. It should ask whether the specialist answer genuinely improves the judgment.

The comparison uses six core factors:

Expert_Superiority = Evidence_Gain + Coverage_Gain + Residual_Reduction + Action_Robustness − Complexity_Cost − Boundary_Risk. (23.5)

This formula should first be used as a qualitative checklist, not as fake numerical precision. The main question is:

Does the expert answer reduce uncertainty, increase coverage, improve action robustness, or provide stronger evidence enough to justify its added complexity? (23.6)

This step corresponds to Sections 10 and 15.

Step 5: Choose one of five governed outcomes

The system should not simply output whichever answer sounds strongest. It should classify the relation between the PORE baseline and the specialist answer.

The five possible outcomes are:

Outcome ∈ {confirm, refine, overturn, residualize, escalate}. (23.7)

Confirm means the specialist answer agrees with the PORE baseline.

Refine means the specialist adds useful detail while preserving the baseline direction.

Overturn means the specialist provides enough evidence to reject the baseline.

Residualize means the disagreement remains meaningful but unresolved, so it becomes a Deviation Residual.

Escalate means the case requires human review, stronger verification, additional evidence, or another specialist universe.

This step corresponds to Sections 10, 11, and 18.

Step 6: Feed unresolved deviations back into knowledge maturation

If the specialist answer and the PORE baseline diverge, and the review cannot fully resolve the difference, the system should not hide that difference inside a polished final answer. It should create a Deviation Residual and feed it back into the knowledge maturation process.

DeviationResidual → KnowledgeMaturationQueue. (23.8)

Over time, these residuals help improve the system. Repeated deviations may reveal that the PORE baseline is too shallow, that the mature knowledge objects are incomplete, that a domain boundary is wrong, or that a new specialist abstraction is needed.

This closes the learning loop.

Runtime disagreement becomes future knowledge improvement. (23.9)

Compact Implementation Formula

The whole method can be compressed into one operational expression:

GovernedAnswer = Synthesize(PORE(K_m, Q, U, Res, Cov), DSS(Q, U, K_m), ExpertReview, Residuals). (23.10)

Or, in an even shorter governance form:

GovernedAnswer = Review(DSS(Q), PORE_Baseline, Res, Cov). (23.11)

The important point is that the final answer is not produced by specialist reasoning alone. It is produced by comparing specialist reasoning against a professional common-sense baseline while preserving residuals and coverage warnings.

Final Institutional Rule

The deepest practical lesson of the framework is this:

Expertise should not abolish common sense. It should be able to defeat it in public. (23.12)

In AI system design, this means:

A specialist answer is not superior because it is more complex, more technical, or more confident. It is superior only when it can show what the common-sense baseline missed, what residual it reduces, what evidence it adds, and why its added complexity is worth accepting.

For implementation, the whole method can be reduced to one operational sequence:

Step 1: Mature the knowledge. (23.1)

Step 2: Generate the PORE baseline. (23.2)

Step 3: Run specialist DSS reasoning. (23.3)

Step 4: Compare specialist answer against baseline. (23.4)

Step 5: Accept, refine, overturn, residualize, or escalate. (23.5)

Step 6: Feed unresolved deviation back into knowledge maturation. (23.6)

In compact form:

GovernedAnswer = Review(DSS(Q), PORE(K_m, Q, U), Res, Cov). (23.7)

And the deepest institutional lesson is:

Expertise should not abolish common sense. It should be able to defeat it in public. (23.8)

A Coarse-Grain Governance Layer for Domain-Specific AI: Knowledge Maturation, Residual Control, and Expert Superiority Review

Part 4 — Appendices, Schemas, Runtime Templates, and End Matter

---

Appendix A — Minimal Object Schemas

This appendix gives practical object schemas for implementing the framework. These are not mandatory database schemas. They are reference contracts. A real system may implement them as JSON, YAML, relational tables, graph nodes, document objects, or internal runtime artifacts.

The important point is not the storage format. The important point is that the runtime should stop treating every intermediate result as loose prose. The system should produce stable objects that can be reviewed, compared, replayed, and improved.

A.1 Raw Knowledge Object

A Raw Knowledge Object is a source-grounded extraction. It should preserve where it came from and avoid premature consolidation.

```
RawKnowledgeObject := {
  object_id,
  source_id,
  source_type,
  source_location,
  extracted_claims,
  local_context,
  provenance,
  uncertainty_notes,
  candidate_domain,
  extraction_method,
  timestamp
}
```

The purpose of this object is not to become final doctrine. It is a disciplined intermediate form.

RawObject ≠ MatureKnowledge. (A.1)

RawObject = source-grounded candidate structure. (A.2)

A.2 Mature Knowledge Object

A Mature Knowledge Object is a consolidated domain object produced under a declared universe.

```
MatureKnowledgeObject := {
  object_id,
  active_universe,
  object_type,
  canonical_statement,
  supporting_raw_objects,
  conflicting_raw_objects,
  evidence_summary,
  domain_scope,
  boundary_conditions,
  residual_packets,
  coverage_status,
  maturity_level,
  update_history,
  reviewer_notes
}
```

The most important fields are:

- active_universe;
- boundary_conditions;
- residual_packets;
- coverage_status;
- maturity_level.

Without these, the object may look mature while remaining epistemically fragile.

MatureObject = consolidated structure + trace + residual + coverage. (A.3)

A.3 Coverage Ledger

The Coverage Ledger records what has been absorbed, what remains outside closure, and what has been explicitly excluded.

```
CoverageLedger := {
  ledger_id,
  active_universe,
  covered_objects,
  partially_covered_objects,
  uncovered_regions,
  excluded_scope,
  stale_regions,
  known_conflicts,
  confidence_summary,
  last_reviewed_at
}
```

Coverage is not merely a count of documents. It is a state of domain closure.

Coverage = covered + partial + uncovered + excluded + stale. (A.4)

A domain answer without coverage status should be treated as weaker than an answer with explicit coverage.

A.4 Residual Packet

A Residual Packet preserves unresolved material.

```
ResidualPacket := {
  residual_id,
  residual_type,
  affected_object,
  active_universe,
  description,
  source_trace,
  severity,
  likelihood,
  action_relevance,
  carry_cost,
  suggested_resolution_path,
  escalation_path,
```

```
  status
}
```

Suggested residual types:

```
residual_type ∈ {
  coverage_gap,
  ambiguity,
  contradiction,
  boundary_conflict,
  verification_gap,
  action_risk,
  novelty_signal,
  translation_gap,
  stale_knowledge,
  ethical_or_governance_risk
}
```

Residual priority can be approximated as:

ResidualPriority = severity × likelihood × action_relevance. (A.5)

The goal is not zero residual. The goal is governed residual.

A.5 PORE Common-Sense Card

The PORE card is the central output of the coarse-grain governance layer.

```
PORE_CommonSenseCard := {
  card_id,
  query_id,
  active_universe,
  situation_summary,

  purpose,
  object,
  residual,
  evaluation,

  baseline_judgment,
  obvious_drivers,
  first_investigation_path,
  red_flags,

  confidence_band,
  coverage_warning,
  maturity_level,

  expert_must_beat,
  generated_from_objects,
  generated_from_residuals,
  timestamp
}
```

The core is:

PORE = Purpose + Object + Residual + Evaluation. (A.6)

The card extends that core into an operational artifact:

PORE_Card = PORE + Baseline + Drivers + RedFlags + ExpertChallenge. (A.7)

A.6 Specialist Answer Object

A Specialist Answer should not be a free-form expert paragraph only. It should state its relation to the PORE baseline.

```
SpecialistAnswer := {
  answer_id,
  query_id,
  specialist_type,
  active_universe,
  input_artifacts,
  tools_used,
  abstractions_used,
  reasoning_summary,
  conclusion,
  evidence_used,
  verification_status,
  residuals_reduced,
  residuals_introduced,
  relation_to_pore_baseline,
  confidence,
  timestamp
}
```

The key field is:

```
relation_to_pore_baseline ∈ {
  confirm,
  refine,
  overturn,
  residualize,
  escalate,
  unrelated_or_wrong_universe
}
```

A.7 Expert Superiority Review Object

This object compares the specialist answer against the PORE card.

```
ExpertSuperiorityReview := {
  review_id,
  card_id,
  specialist_answer_id,

  relation,
  evidence_gain,
  coverage_gain,
  residual_reduction,
  action_robustness,
  complexity_cost,
  boundary_risk,

  superiority_status,
  explanation,
  final_recommendation,
  residuals_to_create,
  escalation_required,
  timestamp
}
```

A simple superiority expression is:

Expert_Superiority = Evidence_Gain + Coverage_Gain + Residual_Reduction + Action_Robustness − Complexity_Cost − Boundary_Risk. (A.8)

Early systems should use qualitative ratings:

Rating ∈ {none, low, medium, high, unknown}. (A.9)

A.8 Deviation Residual Object

When specialist analysis differs from the PORE baseline and the difference is not fully resolved, create a Deviation Residual.

```
DeviationResidual := {
  deviation_id,
  card_id,
  specialist_answer_id,
  deviation_type,
  baseline_statement,
  specialist_statement,
  conflict_summary,
  missing_evidence,
  residual_delta,
  verification_need,
  boundary_risk,
  learning_value,
  suggested_next_action,
  status
}
```

Suggested deviation types:

```
deviation_type ∈ {
  baseline_too_shallow,
  expert_overfit,
  boundary_conflict,
  evidence_asymmetry,
  verification_gap,
  novel_insight_candidate,
  action_risk_conflict
}
```

Deviation = governed difference between common sense and expertise. (A.10)

Appendix B — Runtime Prompt Templates

These templates are written as conceptual prompts. They can be used with any LLM, agent framework, or internal orchestration system. They are deliberately plain so that they can be adapted to a technical runtime.

B.1 PORE Baseline Generator Prompt

```
You are the PORE Coarse-Grain Judgment Layer.

Your task is not to produce the final expert answer.
Your task is to produce a professional common-sense baseline judgment.

Use only the supplied mature knowledge objects, coverage ledger, residual packets, and query context.

Return a PORE_CommonSenseCard with the following fields:

1. Situation Summary
2. Active Universe
3. Purpose
4. Object
5. Residual
6. Evaluation
7. Baseline Judgment
8. Obvious Drivers
9. First Investigation Path
10. Red Flags
11. Confidence Band
12. Coverage Warning
13. Expert Must Beat

Rules:
- Do not hide uncertainty.
- Do not overrule specialists.
- Do not provide a final high-stakes verdict.
- Mark the baseline as provisional if coverage is weak.
- Make the Expert Must Beat field specific.
```

B.2 Specialist DSS Prompt

```
You are a domain-specific specialist system.

You will receive:
1. user query,
2. active universe,
3. mature knowledge objects,
4. symbolic abstractions or tools,
5. PORE Common-Sense Card.

Your task is to produce a specialist answer.

You may confirm, refine, or overturn the PORE baseline.

If you differ from the PORE baseline, you must explain:
1. what the baseline missed;
2. what evidence or mature object changes the judgment;
3. what residual is reduced;
4. what new complexity or risk is introduced;
5. why the deviation is justified.

Return a SpecialistAnswer object.
```

B.3 Expert Superiority Review Prompt

```
You are the Expert Superiority Review layer.

You will receive:
1. PORE Common-Sense Card,
2. Specialist Answer,
3. relevant residual packets,
4. coverage ledger.

Compare the specialist answer against the PORE baseline.

Rate:
- Evidence Gain
- Coverage Gain
- Residual Reduction
- Action Robustness
- Complexity Cost
- Boundary Risk

Then classify the relation as:
confirm, refine, overturn, residualize, escalate, or unrelated_or_wrong_universe.

If the difference is unresolved, create a DeviationResidual proposal.

Return an ExpertSuperiorityReview object.
```

B.4 Final Synthesis Prompt

```
You are the final synthesis layer.

You will receive:
1. PORE Common-Sense Card,
2. Specialist Answer,
3. Expert Superiority Review,
4. Residual Packets,
5. Deviation Residuals if any.
```

```
Produce a final user-facing answer.

Required structure:
1. The professional common-sense baseline
2. What the specialist analysis adds
3. Whether the specialist answer confirms, refines, or overturns the baseline
4. Why the specialist answer is or is not superior
5. Remaining residuals
6. Recommended next action

Do not erase disagreement.
Do not make the answer more certain than the evidence permits.
```

Appendix C — Domain PORE Pack Template

A Domain PORE Pack adapts the PORE kernel to a specific domain.

C.1 Generic Domain Pack

```
DomainPOREPack := {
  domain_name,
  purpose_patterns,
  object_types,
  residual_taxonomy,
  evaluation_criteria,
  obvious_driver_patterns,
  first_investigation_patterns,
  red_flag_patterns,
  evidence_hierarchy,
  escalation_rules,
  maturity_rules,
  review_rules
}
```

The PORE kernel remains constant:

PORE = P + O + R + E. (C.1)

The domain pack changes how each coordinate is interpreted.

Domain_PORE(U) = Interpret(PORE | U). (C.2)

C.2 Evidence Hierarchy Template

Every domain should define what counts as stronger evidence.

```
EvidenceHierarchy := {
  level_1_low: anecdotal or unsupported assertion,
  level_2_basic: source-grounded statement,
  level_3_domain: mature knowledge object or authoritative reference,
  level_4_verified: tool-checked, formally checked, or empirically confirmed,
  level_5_audited: reviewed by domain expert or validated by outcome
}
```

This prevents a specialist answer from winning merely by sounding more technical.

Evidence_Gain requires movement upward in evidence hierarchy. (C.3)

C.3 Escalation Rule Template

```
EscalationRule := {
  trigger,
  reason,
  required_reviewer,
  required_evidence,
  allowed_actions_before_review,
  forbidden_actions_before_review
}
```

Example trigger:

```
trigger = high_residual_severity AND irreversible_action
```

A general rule:

HumanReviewRequired if ResidualSeverity × Irreversibility × BoundaryRisk > Threshold. (C.4)

Appendix D — Worked Example: Enterprise AI Procurement

This example shows the whole method in a business setting.

D.1 Query

A mid-sized company wants to replace its customer-support team with a new AI agent platform. The vendor claims the platform will reduce cost by 60% while maintaining service quality.

D.2 PORE Common-Sense Card

```
Situation Summary:
The company is considering replacing a major customer-support function with an AI agent platform.

Active Universe:
Enterprise operations and AI procurement.

Purpose:
Reduce support cost while preserving service quality, customer trust, regulatory compliance, and operational resilience.

Object:
The proposed AI support platform, the current support workflow, customer interaction types, escalation paths, compliance obligations, and cost structure.

Residual:
Unknown real-world containment rate; unclear failure modes; possible customer dissatisfaction; incomplete data on edge cases; uncertain integration cost; unclear liability for wrong responses.

Evaluation:
Total cost of ownership, customer satisfaction, escalation accuracy, compliance safety, failure recovery, human fallback quality, measurable service-level performance.

Baseline Judgment:
Do not replace the full team immediately. Run a staged pilot with clear escalation rules, human fallback, audit logs, and measured containment quality before committing to full replacement.

Obvious Drivers:
Ticket complexity, customer risk, integration quality, hallucination risk, escalation reliability, workforce transition cost, regulatory exposure.

First Investigation Path:
Classify historical support tickets by risk and complexity; test AI containment on low-risk categories; measure failure and escalation behavior.

Red Flags:
Vendor cannot provide audit logs; no human fallback; no clear escalation policy; high-risk customer categories mixed with low-risk tickets; cost savings assume unrealistic containment.

Confidence Band:
Medium-high as a professional baseline, assuming no mature evidence of exceptional vendor reliability.

Coverage Warning:
Requires company-specific ticket data and pilot results before final decision.

Expert Must Beat:
Any recommendation for immediate full replacement must show reliable containment, low-risk failure behavior, audited escalation, compliance coverage, customer impact evidence, and a realistic total-cost model.
```

D.3 Specialist DSS Answer

The operations DSS may say:

```
The baseline is directionally correct but should be refined.
Historical ticket data shows 72% of tickets are low-risk, repetitive, and resolved through standard scripts.
The recommended path is not full replacement, but category-level automation:
Phase 1: automate password reset, delivery status, refund status, and FAQ tickets.
Phase 2: route billing disputes, complaints, and legal-risk issues to humans.
Phase 3: review performance after 90 days.
```

D.4 Expert Superiority Review

```
Relation:
refine

Evidence Gain:
medium-high; uses ticket category evidence.

Coverage Gain:
medium; covers ticket segmentation but not full customer sentiment impact.

Residual Reduction:
medium; reduces uncertainty by proposing staged categories.

Action Robustness:
high; avoids irreversible full replacement.

Complexity Cost:
low-medium; requires ticket classifier and routing policy.

Boundary Risk:
medium; customer trust and brand impact need separate review.

Outcome:
refine

Final Recommendation:
Adopt staged automation, not full replacement. Create residual packet for customer sentiment and compliance review.
```

D.5 Deviation Residual

```
Deviation Type:
action_risk_conflict

Conflict:
Baseline recommends pilot first; specialist recommends category-level automation with controlled rollout.

Residual Delta:
Specialist reduces operational uncertainty but leaves customer trust and compliance residual.

Learning Value:
medium

Next Action:
Run historical replay test and limited pilot with audit logging.
```

This example shows the practical value of the framework. The PORE baseline prevents reckless adoption. The specialist answer improves the baseline by introducing ticket segmentation. The review prevents the specialist answer from hiding remaining residuals.

Appendix E — Worked Example: Scientific Research Hypothesis

E.1 Query

A research team asks whether a surprising correlation in their data suggests a new causal mechanism.

E.2 PORE Card

```
Purpose:
Determine whether the observed correlation deserves treatment as a causal hypothesis.

Object:
The dataset, observed variables, measurement process, possible confounders, and proposed mechanism.

Residual:
Correlation may be caused by sampling bias, hidden confounder, measurement artifact, multiple testing, or selection effect.

Evaluation:
Replication, causal identification, intervention test, robustness under controls, mechanistic plausibility.

Baseline Judgment:
Treat the result as hypothesis-generating, not causal evidence. Prioritize robustness checks and causal identification before proposing a new mechanism.

Expert Must Beat:
Any causal claim must show why confounding, measurement artifact, and multiple-testing explanations are insufficient.
```

E.3 Specialist Answer

A statistics DSS may refine:

```
The PORE baseline is correct.
The observed correlation disappears after controlling for batch effect and site-level clustering.
The correct conclusion is that the result should not be treated as a causal mechanism.
```

E.4 Review

```
Relation:
confirm with stronger evidence

Evidence Gain:
high

Residual Reduction:
high

Outcome:
confirm

Final Judgment:
The surprising result should be downgraded to an artifact candidate. Create a residual packet for possible measurement bias and update the experimental protocol.
```

This example illustrates a case where specialist analysis does not overturn the baseline. It strengthens it.

Appendix F — Worked Example: Legal-Engineering Conflict

F.1 Query

A company discovers a possible safety defect in a deployed hardware product. Engineering believes the risk is low. Legal believes disclosure may be required.

F.2 Parallel PORE Cards

This is a multi-universe case. The system should not create only one PORE card.

PORE_parallel = {EngineeringCard, LegalCard, OperationsCard, ReputationCard}. (F.1)

F.3 Engineering PORE Card

```
Purpose:
Assess technical safety and failure likelihood.

Object:
The hardware defect, operating conditions, failure rate, severity, detectability, and affected units.

Residual:
Insufficient field data; unclear worst-case operating conditions; uncertain detectability.

Evaluation:
Failure probability, severity, reproducibility, safety margin, test coverage.

Baseline Judgment:
Do not assume low risk until worst-case testing and field data review are complete.
```

F.4 Legal PORE Card

```
Purpose:
Reduce regulatory, liability, and disclosure risk.

Object:
The defect, applicable safety regulations, prior customer incidents, reporting obligations, and internal knowledge timeline.

Residual:
Unclear reporting threshold; incomplete incident record; uncertain regulator interpretation.

Evaluation:
Disclosure duty, liability exposure, documentation quality, timeliness of action.

Baseline Judgment:
Preserve records, investigate immediately, and prepare disclosure analysis before concluding no reporting duty.
```

F.5 Cross-Universe Review

The engineering DSS may say failure probability is low. The legal DSS may say disclosure risk remains high because reporting thresholds depend on severity and knowledge timeline, not only expected failure frequency.

The adjudicator should not average the two.

CrossUniverseDecision = Preserve(EngineeringLowRisk, LegalDisclosureResidual). (F.2)

Final recommendation:

```
Continue technical testing, preserve records, conduct legal disclosure review, and avoid public claims of safety closure until both engineering and legal residuals are reduced.
```

This example shows why PORE must declare active universe. Without universe separation, the system may collapse legal and engineering judgment into a misleading average.

Appendix G — Minimal Implementation in Pseudocode

This appendix gives a simple runtime sketch.

```
function governed_answer(query, active_universe):

    mature_objects = retrieve_mature_objects(query, active_universe)

    coverage = retrieve_coverage_ledger(active_universe, mature_objects)

    residuals = retrieve_residual_packets(query, active_universe, mature_objects)

    pore_card = generate_pore_card(
        query,
        active_universe,
        mature_objects,
        coverage,
        residuals
    )

    specialist_answer = run_specialist_dss(
        query,
        active_universe,
        mature_objects,
        residuals,
        pore_card
    )

    review = expert_superiority_review(
        pore_card,
        specialist_answer,
        coverage,
        residuals
    )

    if review.outcome == "escalate":
        return package_for_human_review(pore_card, specialist_answer, review)

    if review.outcome == "residualize":
        deviation = create_deviation_residual(pore_card, specialist_answer, review)
        enqueue_for_maturation(deviation)

    final_answer = synthesize_final_answer(
        pore_card,
        specialist_answer,
        review,
        residuals
    )

    store_episode_trace(
        query,
        active_universe,
        mature_objects,
        coverage,
        residuals,
        pore_card,
        specialist_answer,
        review,
        final_answer
    )

    return final_answer
```

The governing equation is:

GovernedAnswer = Synthesize(PORE(Q,U,K_m,Cov,Res), DSS(Q,U,K_m), Review, Res). (G.1)

Appendix H — Minimal Database Tables

A simple relational implementation might include:

```
Table: raw_sources
- source_id
- source_type
```

```
- title
- location
- ingestion_time
- metadata

Table: raw_objects
- object_id
- source_id
- extracted_claim
- local_context
- provenance
- candidate_universe
- uncertainty_notes

Table: mature_objects
- mature_id
- active_universe
- object_type
- canonical_statement
- supporting_raw_ids
- conflicting_raw_ids
- boundary_conditions
- maturity_level

Table: residual_packets
- residual_id
- active_universe
- affected_object_id
- residual_type
- description
- severity
- likelihood
- action_relevance
- status

Table: pore_cards
- card_id
- query_id
- active_universe
- purpose
- object
- residual
- evaluation
- baseline_judgment
- expert_must_beat
- maturity_level

Table: specialist_answers
- answer_id
- query_id
- specialist_type
- conclusion
- evidence_used
- verification_status
- relation_to_pore

Table: superiority_reviews
- review_id
- card_id
- answer_id
- evidence_gain
- coverage_gain
- residual_reduction
- action_robustness
- complexity_cost
- boundary_risk
- outcome

Table: deviation_residuals
- deviation_id
- card_id
- answer_id
- deviation_type
- conflict_summary
- learning_value
- status
```

This is enough to start a working prototype.

Appendix I — Practical Adoption Roadmap

I.1 For an individual researcher

Start with:

1. Mature Knowledge Objects for your own writings.
2. PORE cards for each new research question.
3. Specialist analysis from LLMs or tools.
4. Superiority review comparing expert answer against PORE baseline.
5. Deviation residuals stored as future research seeds.

Minimal workflow:

ResearchQuestion -> PORE_Card -> ExpertAnalysis -> SuperiorityReview -> NewResiduals. (I.1)

I.2 For an enterprise team

Start with:

1. one domain universe, such as finance, compliance, support, or engineering;
2. a small mature object library;
3. a PORE card generator;
4. a specialist answer generator;
5. a human review workflow.

Avoid building a full DSS society immediately.

First build the governance surface.

GovernanceSurface first, specialist automation second. (I.2)

I.3 For high-stakes domains

High-stakes domains need stricter controls.

Recommended requirements:

- mandatory active universe declaration;
- mandatory coverage warning;
- mandatory residual disclosure;

- mandatory human review for irreversible actions;
- evidence hierarchy;
- audit trace;
- no final conclusion without review status.

A high-stakes answer should always carry:

FinalHighStakesAnswer = Conclusion + Evidence + Residual + ReviewStatus + HumanResponsibility. (I.3)

Appendix J — Glossary

Active Universe

The declared domain perspective under which knowledge is interpreted. Examples include medicine, law, finance, engineering, compliance, and operations.

Baseline Judgment

The first-pass professional common-sense answer produced by the PORE layer.

Boundary Risk

The risk that the wrong domain, scope, abstraction, or object boundary is being used.

Coverage Ledger

A record of what the knowledge system has covered, partially covered, excluded, or left unresolved.

Deviation Residual

A residual created when specialist analysis diverges from the PORE baseline and the difference itself becomes important.

DSS

Domain-Specific Superintelligence: a specialist AI system designed for deep domain reasoning rather than broad generalist coverage.

Expert Superiority Review

A structured comparison between specialist output and the PORE baseline.

Mature Knowledge Object

A consolidated, traceable, universe-bound knowledge object with residual and coverage status.

PORE

Purpose / Object / Residual / Evaluation. A coarse-grain professional judgment template.

PORE Common-Sense Card

A structured artifact produced by the PORE layer, containing baseline judgment, residuals, red flags, and expert challenge conditions.

Residual

The unresolved remainder under a bounded observer: ambiguity, conflict, missing evidence, fragility, verification gap, or boundary uncertainty.

Specialist Answer

The output of a domain-specific expert system, DSS, tool chain, or human expert.

Superiority

The degree to which specialist analysis improves on the PORE baseline through evidence gain, coverage gain, residual reduction, or action robustness.

Appendix K — Compact Reference Card

K.1 Core idea

Specialist AI should prove why it is better than professional common sense.

K.2 Core pipeline

Raw Sources -> Mature Objects -> DSS Specialists -> PORE Baseline -> Expert Review -> Governed Answer. (K.1)

K.3 Core PORE formula

PORE = Purpose + Object + Residual + Evaluation. (K.2)

K.4 PORE strictness

PORE_Strictness = InputBoundary + OutputSchema + AuthorityLimit + ComparisonSurface + ResidualProtocol. (K.3)

K.5 Expert superiority

Expert_Superiority = Evidence_Gain + Coverage_Gain + Residual_Reduction + Action_Robustness − Complexity_Cost − Boundary_Risk. (K.4)

K.6 Review outcomes

Outcome ∈ {confirm, refine, overturn, residualize, escalate}. (K.5)

K.7 Main governance rule

Expertise should not abolish common sense. It should be able to defeat it in public. (K.6)

End Matter

Final Summary

This paper proposed a coarse-grain governance layer for domain-specific AI. The motivation is that domain-specific specialist systems are necessary but not sufficient. They provide depth, but they still need mature knowledge, residual honesty, and a disciplined comparison surface.

The proposed PORE layer provides that surface. It converts mature domain knowledge into a professional common-sense baseline, then requires specialist answers to show why they confirm, refine, or outperform that baseline.

The result is a new style of AI architecture:

- not monolithic generalist AI;
- not merely a society of specialists;
- not raw RAG over documents;
- not black-box expert synthesis;

but a governed judgment runtime where:

1. knowledge matures before use;
2. residuals remain visible;
3. common sense is made explicit;
4. expertise is tested against a baseline;
5. deviations become learning objects;
6. final answers are reviewable and replayable.

The practical value is simple:

A user should not have to choose between shallow common sense and opaque expert complexity. A good AI system should provide both, compare them, and explain why one should govern the next action.

Reference

- An Alternative Trajectory for Generative AI by Margarita Belova, Yuval Kansal, Yihao Liang, Jiaxin Xiao, Niraj K. Jha, 2026. arXiv:2603.14147 [cs.AI]. https://arxiv.org/abs/2603.14147

- Financial Intelligence & Reasoning Evaluation (FIRE) × Governed Knowledge Objects
https://osf.io/hj8kd/files/osfstorage/69e8f096d1445c7bfefd897d

- Residual Governance for Advanced AI Runtimes From Bounded Observers to Skill Cells, Episode-Time, and Governable Residuals
https://osf.io/hj8kd/files/osfstorage/69e66ce0f69672c19dfd8f03

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.